

Project Proposal

Group: Emma Ross, Matthew Rossi, Michael Xiao, Shu Luan

1 Motivation

In an idealized ranked choice election, voters rank all candidates from most preferred to least preferred. With this collection of rankings, called a *preference profile*, in hand, a winner is crowned according to some preordained rule. How well does that winner satisfy society? Suppose that each voter v has a utility function $u_v : \mathcal{C} \rightarrow \mathbb{R}$, which assigns a utility $u_v(c)$ to each candidate $c \in \mathcal{C}$ (the set of candidates), representing how good candidate c is for v personally. Then, he simply ranks candidates in order of decreasing utility. Although challenging given the limited information present in rankings, we might wish to choose the candidate that maximizes the sum of voter utilities.

Where do these utilities come from? One strong but useful assumption frequently made in the computational social choice (COMSOC) literature is that candidates and voters are jointly embedded in some metric space, and that a voter assigns a *cost* to each candidate according to his distance from that candidate. Thus, a voter prefers candidates closer to him over those farther away.

If we place voters and candidates in two dimensions by sampling from some distribution, this generates a unique preference profile (the reverse is not true: a given preference profile is consistent with many distinct embeddings). Running various voting rules on this preference profile and plotting the winners can give visual intuition for the behavior of the voting rules. Using this approach for multi-winner rules, [Elkind et al.](#) demonstrated that some voting rules (like k -Borda) tend to elect a committee of centrists; others (such as STV) elect committees that are representative, i.e., they look like the voter distribution; while others (like Bloc Plurality) exhibit a [center squeeze](#), electing a committee of extreme candidates.

The core objective of our project is to take this spatial approach to simulating elections and make it *interactive*. Allowing users to build, experiment, and iterate upon their own simulations builds intuition in ways that a set of static plots cannot.

The intended audience for our web application is threefold:

1. COMSOC researchers can experiment with various voting rules, testing and refining the intuition they already have for those methods. Visualization may also suggest behavior patterns in these voting rules that can be axiomatized and studied formally.
2. Students studying social choice theory can use the tool to develop intuition for the behavior of voting rules that supplements their learnings in the classroom. More aspirationally, perhaps they might be inspired to pursue research in the area.
3. Policymakers and policy advocates who are considering the implementation of ranked choice voting and wondering what voting rule to use might find value in the tool as they develop intuition for the behavior of and tradeoffs between the rules under consideration.

2 Technical Specifications

A disclaimer: the user experience specification we have in mind is ambitious given the timeframe for the project. We reserve the right to overpromise and underdeliver; some of these features may be cut, or added at a later date.

Upon loading, the user is presented with a blank scatterplot with no axes. A message inside the plot prompts the user to place their first voter. They can opt for one of four methods for creating spatial plots of voters and candidates:

1. **Manual Input** (default) - Below the plot is an option for the user to select whether they are placing voters or candidates. Then, they left click to place a point on the plot. Voters are circles, while candidates are stars. Left clicking on an existing point will delete it. Right clicking an existing point will allow the user to switch it from a voter to a candidate (or vice versa) or change the number of voters at that particular point. Manual input is always available, including when using the generative methods—for example, a user may sample voters programmatically and place some candidates by hand.

2. ***Synthetic Sampling*** - Selecting this option causes a side panel to appear or populate. The panel contains a distribution list, which is empty at first. At the bottom of the panel is a *generate* button. The user can add a distribution, which has the following fields: type of distribution; whether to sample voters, candidates, or both from this distribution; number of voters/candidates to sample; and distribution parameters. Supported distributions include:
 1. **Gaussian** - select “center” and then click the plot to set the center at that point. Type a number to set the spread/variance.
 2. **Uniform Rectangle** - defaults to covering the entire plot. Otherwise, can select “center” and set the center at that point, then type numbers to set the length and width.
 3. **Uniform Disk** - select “center” and then click the plot to set the center at that point. Type a number to set the diameter.
3. ***Survey Data*** - Selecting this option causes a side panel to appear or populate. The panel contains a map of Illinois, where the user can choose their jurisdiction: state level or county-specific. The panel contains several options for “ideology” or “issue” axes. The user shall select two of them. There is a checkbox labeled “Indicate Party Affiliation?”. If checked, the generated voters will be colored blue, red, or gray according to their party affiliation; otherwise all voters will be the same color. Clicking “generate” samples voters from an ideology distribution inferred from survey data, and the axes are labeled correspondingly.
4. ***Data Upload*** - If a user has exported a JSON file of the points from a previously-generated plot, they may upload that data.

The following interactions are available:

1. ***Highlight on hover*** - hovering over a point in the plot gives it a dark outline, and, if it is a voter point with more than one voter, displays the number of voters at that point.
2. ***Export plot*** - plot can be exported as an image or as a JSON file for later re-import.
3. ***Generate*** - Clicking “generate” from either the Synthetic Sampling or Survey Data tabs clears the existing plot data and generates anew. First time clicking in a session will warn the user that this action cannot be undone.
4. ***Clear plot*** - Requires confirmation (every time), as this action cannot be undone.

Running elections and producing results:

1. ***Setting election settings*** - user chooses the voting rule to run and the number of winners.
2. ***Run election*** - creates/updates the following displays:
 1. The plot is updated to display the winner in a different color, and decreases the opacity of losing candidates.
 2. The plot is updated to display the candidate with the lowest social cost (the “optimal” candidate).
 3. A separate visualization shows the score of each candidate (for voting rules that produce a meaningful score for each candidate). This visualization plots only the candidates from the spatial plot, and their sizes are scaled according to their scores. These “candidate bubbles” push each other out of the way so as to be non-overlapping. Displays 10 candidates maximum.

3 Group Member Assignments

- **Emma Ross** - Data sourcing, aggregating, and cleaning for survey-based simulations. Identification of feasible “ideology”/“issue” axes. Design and implementation of methods for inferring distribution of voters across ideological or issue-based spectra. API design for the survey-based simulation functionality.
- **Matthew Rossi** - Frontend, visualization, and interaction requirements specification. UX design. Backend implementation with [VoteKit](#).
- **Michael Xiao** - UI design of web app and visualization to optimize for clarity. Front-end implementation of visualization and tool designs.
- **Shu Luan** - API design and data format specification for communication and data-transfer between JS frontend and Python backend. Assistance with backend or frontend implementation as needed.